

2. Analyze the two GenAI programs.

- a. Name of two GenAI programs
 - i. Google Gemini flash 3.7 high
 - ii. Claude-Sonnet 4.6
- b. How you accessed them (e.g., Google query, CoPilot on my computer)
 - i. I used Google's AI harness Antigravity to access both models
- c. Prompt used. (You must use the same prompt for both GenAIs.)
 - i. I used these two images and this text as my prompt:
 - ii. Generate this program using the following specifications exactly:

Assignment:

1. Use two GenAI's to generate C code to implement the following requirements
(Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI):
 - The program will prioritize emails for a busy company CEO.
 - You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
 - You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
 - You must create functions from scratch. Do not include pre-existing heap modules.

• Test File format:

Command	Description
EMAIL <sender category>,<subject line>,<date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- d. Generated code from both GenAIs. (the files are too long to copy here)
 - i. Claude: claude_sonnet_4.6.c
 - ii. Gemini: gemini_initial.c (I edited the original since I picked it to improve, so this is a copy of what Gemini made on the first prompting)
- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.
 - i. Correctness
 1. Claude: The program runs properly when given any valid sample input. Whenever there is a line that the program doesn't understand, can't interpret, or the command is unfinished, it just skips over it (silently fails). When passed in incorrect input, like an invalid date or sender or whatever it will happily print them, but it will always put any non-defined sender at

the bottom of the queue. If an invalid file name is given, it will print a formatted error message saying "Error: cannot open (input)".

2. Gemini: Gemini has similar behaviour, with any well formatted input properly printing the output, except this time there are no newlines between the outputs. This isn't as correct as Claude, since the assignment did ask for that formatting (or at least the sample output was given in that format). Gemini's code has an error message for file not found. It also has the same silent failing that Claude did, where whenever something in the file didn't compute it would skip it. It also accepts invalid dates and senders, like Claude does. Really, both are decently correct but are all too willing to accept logic-error stuff and not say anything.

ii. Execution Time / Time complexity

1. For time complexity in $O(n)$ terms, both programs are exactly the same. Both programs implement a max heap as a list (array), and when both are upheaping, downheaping, heapifying, or whatever you want to call it with a new email, they both are operating on $O(\log(n))$ time or $O(1)$ time. However, when it comes to actual execution time, the gemini program is roughly 3% faster. I had my IDE agent create large sample files of multiple thousands of lines to run benchmarks and then compare the results between the two files. Both programs are almost entirely alike in actual implementation, with Gemini's extra lines of code mostly being more verbose, more easily readable ways of doing what Claude is also doing. The noticeable difference is entirely found in the $\ln$ that Claude prints along with every output. That singular extra step, after hundreds of thousands of commands, eventually adds up, giving Claude a slight delay compared to Gemini.

iii. Space Time / Space Complexity

1. Big picture: both programs are $O(n)$ space complexity for gobbling up space, with n being the maximum number of unread emails ever held in the heap. When it comes to runtime memory used, they are also entirely similar. Both implementations of the MaxHeap use a struct and Email as a struct, where both programs strictly define how much memory each member will take up. All memory constants like, for example, `MAX_SENDER_LEN` are the same as their counterparts in both programs, meaning each is allocated the same memory when running. However, there is a very slight discrepancy in memory used at runtime, where Claude used roughly .017% less memory than Gemini, which was about 23kb. This is because of the string literals in the programs. Claude has more reused literals "Error: out of memory" is reused 3 times, so the compiler can just reuse it, while Gemini has a different error message for each of the 3 memory failure points. This can be directly seen in the actual file sizes! For me, the compiled Gemini code is 118kb, while Claude's compiled code is 116kb!

iv. Maintainability

1. By FAR, Gemini's program is a much more maintainable program. It has clear function names compared to Claude, like `compare_emails(*args)` vs `cmp(*args)`. Gemini also avoided doing the single line if-statements as much as Claude did. I don't know why Claude likes them so much, but they are sort of dangerous and a little unclear to a reader, since I like my logic to be chunked out and all nice to read. Gemini also had more verbose errors than Claude did when it came to memory errors. Claude used the same memory error 3 times "Error: out of memory" while Gemini described what had happened "Error: Memory allocation failed for MaxHeap structure.\n". Gemini also used more comments. Basically Gemini absolutely swept this round, and Claude is never beating the Claudish curse of being indecipherable. Just to see what happened, I asked my agent if Claude did anything better than Gemini, and it did say Encapsulation was done better. When prototyping the functions out, Claude declared each one as static, which makes them invisible to any other potential files in the project. If another file had some other heap with the same function names, Gemini's program would cause problems, while Claude's wouldn't, making the project easier to approach and less coupled together.
- f. Select one to use as the basis for this assignment and justify why you picked it.
- i. I am picking Gemini's code, simply for the reason that it is more approachable, understandable, and readable. It is more approachable because of the comments available and the way the variables and functions are named. It is more understandable due to the style the functions and statements are written in (little multi-statement single lines). It is more readable due to much of the same, with the overall structure being more familiar to me than the Claudish. I also feel I can also learn more by editing Gemini's code, since Claude's seems so high level that to properly learn from it, I should already know C.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
- i. Correctness:
 1. I improved the correctness of Gemini's program (Gemini_3.7.c) by giving the program date checking code. This ensures input must be valid, and you won't be getting emails on February 31st. I also added command validation, so if an invalid command is put in (like SEARCH or anything that isn't 1 of the 4 commands) it will fail and exit. I also changed every if-statement to use {}, since I googled it and industry standards across the board mandate such things to avoid, as I found out, something similar to Apple's goto fail; goto fail; bug. Kind of cool and upsetting that one small line of code can just wreck havoc like that on a whole OS.
 - ii. Execution Time and Spatial Complexity:

1. The program was almost another example of me only hurting time and space complexity, since I added more input-validation like my date checking and command checking, but I did improve both the time and spatial complexity! The original implementation of the MaxHeap used an array of Emails, but now it uses an array of *Emails (Email pointers), which cut down the struct's space by 97% (I asked my agent to run a memory test)! This means whenever data is written to or removed from the MaxHeap it happens faster since it is removing and adding simple memory addresses and not whole structs of 348 bytes. I also improved (in more broad terms) the spatial complexity of the program by dynamically shrinking the memory allocated for the array when Emails are removed. This is just more memory-safe, and it allows for the program to not have dead space left over from when the array peaked in size, which is good for your computer!

iii. Maintainability:

1. I improved maintainability in a few simple but significant ways. I made sure variable names were clear and understandable (at least enough to be read by someone familiar with the language and purpose). I also added comments to EVERY LINE, so now there is a play-by-play of what is happening in the program. This did, in a way, hurt maintainability slightly by doubling the number of lines, but it was more beneficial than detrimental! I also spaced out cluttered statements with a line in between statements in the same scope, since I think it makes it look nicer.